



DOCKER

Complete Reference Guide

Zero to Advanced · CLI Commands · Internals · Interview Prep

Source: docs.docker.com | Senior DevOps Engineer Edition



Table of Contents

 Table of Contents.....	2
1. FOUNDATION LEVEL	6
1.1 What is Docker?	6
1.2 Why Docker Exists — The Problem It Solves	6
1.3 Containers vs Virtual Machines	7
1.4 Core Docker Architecture	7
1.5 Docker Installation	8
Windows	8
macOS	8
Linux (Ubuntu / Debian)	9
2. CORE CONCEPTS	10
2.1 Images	10
Image Layers	10
2.2 Containers	11
Container Lifecycle	11
2.3 Dockerfile	12
2.4 Layers	13
2.5 Volumes	13
2.6 Networks	14
2.7 Bind Mounts	15
2.8 Environment Variables	15
2.9 Ports & Exposure	16
2.10 Registry — Docker Hub & Private Registries	16
3. WORKFLOW & COMMANDS	18
3.1 Image Lifecycle	18
3.2 Container Lifecycle	18
3.3 Essential CLI Quick Reference	18
System Commands	18
docker run — Most Used Flags	19
3.4 Docker Logs	19
3.5 Docker Inspect	19
3.6 Exec vs Attach	20
3.7 Prune Commands — Cleaning Up	20
4. DOCKERFILE DEEP DIVE	22
4.1 All Major Instructions	22

FROM — Base Image	22
RUN — Execute Commands During Build	22
CMD — Default Runtime Command	22
ENTRYPOINT — Fixed Executable	23
COPY — Copy Files from Build Context	23
ADD — Copy with Extras	23
WORKDIR — Set Working Directory	23
ENV and ARG	23
EXPOSE, VOLUME, HEALTHCHECK, USER	24
4.2 Multi-Stage Builds	24
4.3 Build Cache Behaviour	25
4.4 Best Practices & Security	26
Official Best Practices	26
Security	26
5. DOCKER NETWORKING	27
5.1 Network Drivers Overview	27
5.2 Bridge Network	27
5.3 Host Network	27
5.4 Overlay Network (Swarm)	28
5.5 Macvlan Network	28
5.6 Container-to-Container Communication	29
5.7 How DNS Works Inside Docker	29
6. DOCKER STORAGE	30
6.1 Storage Types Overview	30
6.2 Named Volumes in Depth	30
Volume Backup & Restore	31
6.3 Bind Mounts in Practice	31
6.4 Volume Drivers	31
6.5 Data Persistence Best Practices	32
7. DOCKER COMPOSE	33
7.1 What Problem Does Compose Solve?	33
7.2 YAML Structure Breakdown	33
7.3 Service Configuration Options	34
7.4 Essential Compose Commands	34
7.5 Complete Example — Full-Stack Application	35
7.6 Compose V2 Features	36
8. DOCKER SECURITY	37
8.1 Linux Namespaces	37
8.2 Control Groups (cgroups)	37

8.3	Image Scanning	37
8.4	Least Privilege Principle	38
8.5	Rootless Mode	38
8.6	Docker Bench for Security	38
8.7	Docker Content Trust (DCT).....	39
8.8	Security Checklist.....	39
9.	DOCKER IN PRODUCTION	40
9.1	Logging Drivers	40
9.2	Resource Limits.....	40
9.3	Restart Policies	41
9.4	Healthchecks.....	41
9.5	Scaling Concepts	42
9.6	Docker Swarm Basics	42
10.	ADVANCED TOPICS.....	44
10.1	BuildKit.....	44
10.2	Docker Swarm — Extended	44
	Service Types.....	44
10.3	Private Docker Registry	45
10.4	Docker Context.....	45
10.5	Docker Extensions	46
10.6	Docker Desktop Architecture	46
11.	TROUBLESHOOTING	47
11.1	Common Errors & Fixes	47
	Error: "port is already allocated"	47
	Error: "no space left on device"	47
	Error: "image not found" / "manifest unknown"	47
	Error: Container exits immediately	47
	Error: "Cannot connect to the Docker daemon".....	48
	Error: Build context too large.....	48
	Error: Containers cannot communicate.....	48
	Error: exec format error.....	48
11.2	Debugging Workflow	49
11.3	Performance Optimisation Tips	49
	Image Size.....	49
	Build Speed	50
	Runtime Performance.....	50
12.	INTERVIEW PREPARATION.....	51
12.1	Beginner-Level Questions	51
	Q1: What is Docker and why do we use it?	51

Q2: Containers vs Virtual Machines?	51
Q3: What is a Dockerfile?	51
Q4: Difference between CMD and ENTRYPOINT?.....	51
Q5: COPY vs ADD?.....	51
Q6: What are Docker volumes?	51
12.2 Intermediate-Level Questions.....	51
Q7: Explain Docker networking — what are the drivers?	51
Q8: What are layers and how does caching work?	52
Q9: docker stop vs docker kill?	52
Q10: What is a multi-stage build?	52
Q11: How do containers communicate?	52
Q12: What is Docker Compose?.....	52
12.3 Advanced-Level Questions.....	52
Q13: How does Docker achieve isolation?.....	52
Q14: How would you troubleshoot a container that keeps restarting?	52
Q15: How do you secure Docker in production?	52
Q16: What is BuildKit?	53
12.4 Scenario-Based Problems	53
Scenario 1: App works locally, crashes in Docker	53
Scenario 2: Container killed with exit code 137	53
Scenario 3: Docker build is very slow.....	53
Scenario 4: Data lost after container restart	54
Scenario 5: High CPU usage — diagnosis.....	54
13. 30-DAY MASTERY ROADMAP & REFERENCES	55
30-Day Docker Mastery Roadmap	55
Week 1 — Foundation (Days 1–7).....	55
Week 2 — Networking & Compose (Days 8–14)	55
Week 3 — Production & Security (Days 15–21).....	55
Week 4 — Advanced & Interview (Days 22–30)	56
Practical Project Ideas.....	56
Official Documentation Links	56
Getting Started	56
Core References.....	56
Building.....	57
Networking, Storage & Security	57
Production & Orchestration	57
Quick Reference Card.....	57

1. FOUNDATION LEVEL

1.1 What is Docker?

Simple Explanation:

Docker is a tool that lets you package your application along with everything it needs (code, libraries, settings) into a single portable unit called a container. That container runs the same way on any machine — your laptop, a colleague's laptop, or a production server.

i Official Definition: "Docker is an open platform for developing, shipping, and running applications. Docker enables you to separate your applications from your infrastructure so you can deliver software quickly." — docs.docker.com

Real-World Analogy:

Think of Docker like a shipping container in global trade. Before shipping containers, goods were loaded loosely onto ships — fragile, inconsistent, slow. Shipping containers standardised everything: any crane can lift them, any truck can carry them, any ship can hold them. Docker does the same for software.

1.2 Why Docker Exists — The Problem It Solves

The "Works on My Machine" Problem:

- Developer A: "The app works perfectly on my machine!"
- Developer B: "It crashes on mine..."
- Ops Team: "It doesn't even start in production..."

Root causes: different OS versions, different library versions, missing dependencies, different environment variables, different file paths.

BEFORE DOCKER		
Dev Machine	Staging Server	Prod Server
Python 3.9	Python 3.8	Python 3.7 ← GAP
Ubuntu 20.04	CentOS 7	Amazon Linux ← GAP

AFTER DOCKER		
Dev Machine [Container]	Staging Server [Container]	Prod Server [Container]
Python 3.9	Python 3.9	Python 3.9 ✓ SAME
Ubuntu 22.04	Ubuntu 22.04	Ubuntu 22.04 ✓

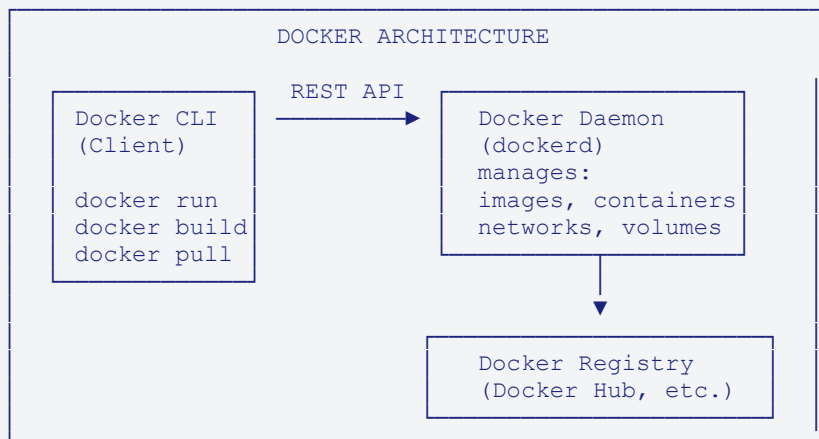
1.3 Containers vs Virtual Machines

VIRTUAL MACHINES			CONTAINERS		
App A	App B	App C	App A	App B	App C
Libs	Libs	Libs	Libs	Libs	Libs
Guest OS	Guest OS	Guest OS	Docker Engine		
Hypervisor			Host OS		
Infrastructure			Infrastructure		
Startup: MINUTES			Startup: SECONDS		
Size: GIGABYTES			Size: MEGABYTES		

Feature	Virtual Machine	Container
OS	Each VM has its own full OS	Shares host OS kernel
Size	Gigabytes	Megabytes
Startup time	Minutes	Seconds
Isolation	Strong — hardware level (hypervisor)	Process-level (namespaces)
Performance	Near-native with overhead	Near-native (minimal overhead)
Portability	Less portable	Highly portable
Resource usage	Heavy	Lightweight
Use case	Full OS isolation needed	Microservices, CI/CD, scaling

Analogy: VM = each app gets its own house (full OS). Container = each app gets its own apartment in a shared building (shared kernel, isolated space).

1.4 Core Docker Architecture



Component	Role
Docker Client	CLI tool (docker command). Sends commands to the daemon via REST API
Docker Daemon (dockerd)	Background service that manages images, containers, networks, volumes
Docker Engine	The client + daemon + REST API together
Docker Registry	Stores Docker images. Docker Hub is the default public registry
Docker Desktop	GUI app for Mac/Windows bundling Engine + CLI + Compose + Kubernetes

Verification Commands

```
docker --version      # Quick version check
docker version        # Full client + server version info
docker info           # System-wide info: containers, images, storage driver
docker context ls      # List Docker contexts (local, remote)
docker system info     # Detailed system information
docker run hello-world # End-to-end verification test
```

1.5 Docker Installation

Windows

```
# 1. Enable WSL 2
wsl --install

# 2. Download Docker Desktop from:
#   https://docs.docker.com/desktop/install/windows-install/

# 3. Verify
docker --version
docker run hello-world

# Requirements: Windows 10/11 64-bit, WSL 2, BIOS virtualisation enabled
```

macOS

```
# 1. Download Docker Desktop from:
#   https://docs.docker.com/desktop/install/mac-install/
# 2. Drag Docker.app to Applications, open it
# 3. Verify:
docker --version
docker run hello-world
```

Linux (Ubuntu / Debian)

```
# Remove old versions
sudo apt-get remove docker docker-engine docker.io containerd runc

# Add Docker GPG key and repository
sudo apt-get update
sudo apt-get install ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

# Install Docker Engine
sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin

# Run without sudo (add user to docker group)
sudo usermod -aG docker $USER
# Log out and back in, then verify:
docker run hello-world
```

2. CORE CONCEPTS

2.1 Images

Simple Explanation:

An image is a blueprint/template for creating containers. It contains your app code, libraries, dependencies, and configuration — everything needed to run. Images are read-only.

i Official: "A Docker image is a read-only template with instructions for creating a Docker container. Often, an image is based on another image with some additional customization." — Docker Docs

Real-World Analogy: An image is like a recipe. You can use the same recipe to cook the dish many times (create many containers), but the recipe itself never changes.

Image Layers

Layer 5: CMD ["python"...]	← Metadata only
Layer 4: COPY . .	← Your code
Layer 3: RUN pip install	← Dependencies
Layer 2: COPY requirements.txt	← requirements.txt
Layer 1: FROM python:3.11-slim	← Base image

(all layers read-only, cached)

CLI Commands

Command	Description
<code>docker pull nginx</code>	Pull image from Docker Hub (latest tag)
<code>docker pull python:3.11-slim</code>	Pull specific version tag
<code>docker images</code>	List local images (alias: <code>docker image ls</code>)
<code>docker rmi nginx</code>	Remove image (alias: <code>docker image rm nginx</code>)
<code>docker build -t myapp:1.0 .</code>	Build image from Dockerfile in current dir
<code>docker build -f Dockerfile.dev .</code>	Build with a specific Dockerfile name
<code>docker build --no-cache -t app .</code>	Build ignoring all cached layers
<code>docker tag myapp:1.0 user/myapp</code>	Create alias/tag for image
<code>docker push user/myapp:1.0</code>	Push image to registry
<code>docker image inspect nginx</code>	Full JSON details of image
<code>docker image history nginx</code>	Show all layers and their sizes
<code>docker save -o myapp.tar myapp:1.0</code>	Export image to tar file
<code>docker load -i myapp.tar</code>	Import image from tar file
<code>docker image prune</code>	Remove dangling (untagged) images

Command	Description
<code>docker image prune -a</code>	Remove ALL unused images

⚠ Common Mistakes: Not specifying a tag (defaults to :latest — unpredictable in production). Using large base images (python:3.11 at 900MB) instead of slim/alpine variants. Not running `docker image prune` — disk fills up.

2.2 Containers

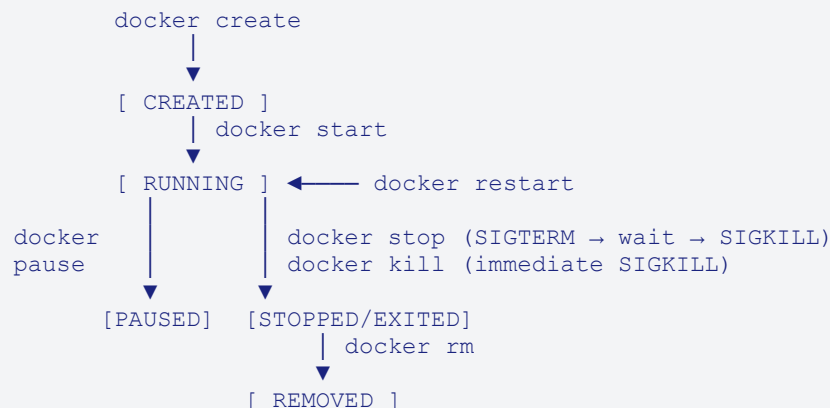
Simple Explanation:

A container is a running instance of an image. If an image is a recipe, a container is the actual dish cooked from that recipe. You can run multiple containers from the same image — each is independent.

i Official: "A container is a runnable instance of an image. You can create, start, stop, move, or delete a container using the Docker CLI or API." — Docker Docs

Real-World Analogy: Image = Class in programming. Container = Object (instance of that class).

Container Lifecycle



CLI Commands

Command	Description
<code>docker run -d nginx</code>	Create + start container in background
<code>docker run -it ubuntu bash</code>	Interactive terminal session
<code>docker run --name web nginx</code>	Assign a custom name
<code>docker run --rm nginx</code>	Auto-remove container when it stops
<code>docker run -d -p 8080:80 nginx</code>	Map host port 8080 → container port 80
<code>docker run -e PORT=3000 myapp</code>	Set environment variable
<code>docker run -v mydata:/app myapp</code>	Mount named volume
<code>docker run --cpus="1.5" myapp</code>	Limit CPU usage
<code>docker run --memory="512m" myapp</code>	Limit memory usage

Command	Description
<code>docker create --name web nginx</code>	Create without starting
<code>docker start web</code>	Start a created/stopped container
<code>docker stop web</code>	Graceful stop (SIGTERM then SIGKILL)
<code>docker kill web</code>	Immediate stop (SIGKILL only)
<code>docker restart web</code>	Stop then start
<code>docker rm web</code>	Remove stopped container
<code>docker rm -f web</code>	Force remove (even if running)
<code>docker ps</code>	List running containers
<code>docker ps -a</code>	List ALL containers including stopped
<code>docker logs -f web</code>	Follow container logs in real time
<code>docker exec -it web bash</code>	Open shell inside running container
<code>docker stats</code>	Live CPU/memory usage for all containers
<code>docker top web</code>	Processes running inside container
<code>docker cp file.txt web:/app/</code>	Copy file into container
<code>docker rename old new</code>	Rename a container

⚠ Common Mistakes: Forgetting `-d` runs container in foreground (blocks terminal). Using `docker run` repeatedly on stopped containers (creates duplicates — use `docker start`). Storing data inside container (lost on `docker rm`).

2.3 Dockerfile

A Dockerfile is a text file with step-by-step instructions to build an image. Like a recipe card that Docker follows to create your image.

i Official: "A Dockerfile is a text document that contains all the commands a user could call on the command line to assemble an image." — Docker Docs

```
# Minimal best-practice Dockerfile
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .          # Copy deps first (cache optimisation)
RUN pip install --no-cache-dir -r requirements.txt

COPY . .                        # Copy app code after

EXPOSE 5000
CMD ["python", "app.py"]
```

Build Commands

Command	Description
<code>docker build -t myapp:1.0 .</code>	Build with name:tag from current dir
<code>docker build -f Dockerfile.dev .</code>	Build using specific Dockerfile
<code>docker build --no-cache -t app .</code>	Build with no cache
<code>docker build --build-arg K=V .</code>	Pass build argument
<code>docker build --target stage .</code>	Build only to named stage (multi-stage)
<code>docker builder prune</code>	Clean build cache
<code>docker buildx build ...</code>	Build with BuildKit (multi-platform etc.)

⚠ Mistake: Putting COPY . . before RUN pip install — breaks build cache on every code change. Always copy dependency files first, then install, then copy application code.

2.4 Layers

Every Dockerfile instruction creates a layer. Layers are stacked, read-only, and cached. Docker reuses unchanged cached layers, making rebuilds fast.

✓ Key rule: Place instructions that change LEAST FREQUENTLY first. Put frequently-changing code copies LAST. This maximises cache hits.

```
# BAD - cache busted on every code change
COPY . .                ← Any file change invalidates everything below
RUN pip install -r req.txt ← Re-runs on EVERY code change (slow!)

# GOOD - pip install cached until requirements.txt changes
COPY requirements.txt .   ← Only changes if requirements.txt changes
RUN pip install -r req.txt ← Cached until requirements change
COPY . .                 ← Code changes don't affect pip cache
```

Command	Description
<code>docker image history myapp:1.0</code>	Show all layers with sizes and commands
<code>docker image inspect myapp:1.0</code>	Full JSON including layer SHA256 hashes
<code>docker build --no-cache .</code>	Build ignoring all cached layers
<code>DOCKER_BUILDKIT=1 docker build .</code>	Enable BuildKit for better caching

2.5 Volumes

Volumes are Docker's way of storing data that persists even after a container is deleted. Docker manages them — stored in `/var/lib/docker/volumes/`.

i Official: "Volumes are the preferred mechanism for persisting data generated by and used by Docker containers. Volumes are completely managed by Docker." — Docker Docs

Real-World Analogy: A volume is like a locker at the gym. You (the container) may come and go, but your stuff in the locker stays.

Command	Description
<code>docker volume create mydata</code>	Create a named volume
<code>docker volume ls</code>	List all volumes
<code>docker volume inspect mydata</code>	Inspect volume details and host path
<code>docker volume rm mydata</code>	Remove a specific volume
<code>docker volume prune</code>	Remove all unused volumes (DANGEROUS)
<code>docker run -v mydata:/app nginx</code>	Mount named volume (short syntax)
<code>docker run --mount type=volume,...</code>	Mount volume (explicit --mount syntax)

⚠ Critical Mistake: Not using volumes for databases. Data stored in the container layer is permanently lost when you run `docker rm`. Always use `-v mydata:/path` for any data you care about.

2.6 Networks

Docker networks allow containers to communicate. Containers on the same custom network resolve each other by name via Docker's built-in DNS server.

i Official: "Docker's networking subsystem is pluggable using drivers. Several drivers exist by default and provide core networking functionality." — Docker Docs

Command	Description
<code>docker network ls</code>	List all networks
<code>docker network create mynet</code>	Create custom bridge network
<code>docker run --network mynet nginx</code>	Run container on specific network
<code>docker network connect mynet c1</code>	Connect running container to network
<code>docker network disconnect mynet c1</code>	Disconnect container from network
<code>docker network inspect mynet</code>	Inspect network details and connected containers
<code>docker network rm mynet</code>	Remove a network
<code>docker network prune</code>	Remove all unused networks

⚠ Mistake: Using the default bridge network. It has NO DNS — containers can only communicate by IP address. Always create custom bridge networks so containers can reach each other by name.

2.7 Bind Mounts

Bind mounts map a specific folder on your host machine to a folder inside the container. Changes on either side are reflected immediately — ideal for development.

Real-World Analogy: A bind mount is like sharing a Google Doc — changes by anyone appear instantly for everyone.

```
# Short -v syntax
docker run -d -v /path/on/host:/app nginx

# Explicit --mount syntax (recommended for scripts)
docker run -d \
  --mount type=bind,source=$(pwd)/src,target=/app/src \
  myapp:dev

# Read-only bind mount
docker run -d -v $(pwd)/config:/app/config:ro nginx

# Windows example
docker run -d -v C:\Users\me\project:/app nginx
```

Feature	Named Volume	Bind Mount
Managed by	Docker	User / OS
Location	/var/lib/docker/volumes/	Any host path you specify
Portability	High — works on any Docker host	Low — tied to host path structure
Pre-populated from image	Yes	No — overwrites container dir
Best for	Production, databases	Development, live code reload

2.8 Environment Variables

Environment variables let you pass configuration to containers at runtime without changing the image. Essential for passwords, API URLs, feature flags.

Real-World Analogy: Like a thermostat on your oven — the recipe (image) is the same, but you adjust the temperature (env vars) for each deployment.

```
# Single variable
docker run -e MY_VAR=hello nginx

# Multiple variables
docker run -e DB_HOST=db -e DB_PORT=5432 myapp

# Load from .env file
docker run --env-file .env myapp

# In Dockerfile — sets default (overridable at runtime)
ENV APP_ENV=production
```

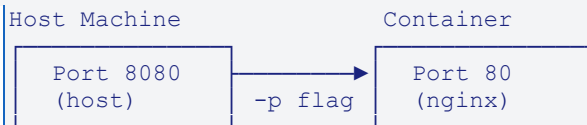
```
ENV PORT=8080

# Inspect env vars in running container
docker exec myapp env
docker inspect --format="{{range .Config.Env}}{{println .}}{{end}}" myapp
```

⚠ Critical: Never hardcode secrets in Dockerfile ENV instructions — they are baked into the image and visible via docker inspect. Use --env-file or Docker Secrets.

2.9 Ports & Exposure

Containers are isolated by default. To access a service inside a container from the outside, you must explicitly publish (map) ports.



Command	Description
<code>docker run -p 8080:80 nginx</code>	Map host 8080 → container 80
<code>docker run -p 8080:80 -p 443:443</code>	Map multiple ports
<code>docker run -p 127.0.0.1:8080:80</code>	Map only on localhost interface
<code>docker run -P nginx</code>	Publish ALL EXPOSE'd ports to random host ports
<code>docker port mycontainer</code>	View port mappings of a container

⚠ EXPOSE in Dockerfile is documentation only — it does NOT publish the port. You still need -p at runtime. Thinking EXPOSE makes the port accessible is the most common beginner mistake.

2.10 Registry — Docker Hub & Private Registries

A registry is a storage and distribution system for Docker images. Docker Hub is the default public registry. Private registries store proprietary images securely.

i Official: "A Docker registry stores Docker images. Docker Hub is a public registry anyone can use, and Docker looks for images on Docker Hub by default." — Docker Docs

```
# Image naming format: [registry/][user/]image[:tag]

nginx                → docker.io/library/nginx:latest
myuser/myapp:v1.0    → docker.io/myuser/myapp:v1.0
registry.co/team/app:v2 → private registry

# Login
docker login          # Docker Hub
docker login registry.company.com # Private registry

# Pull / Push
docker pull nginx:1.25.3
docker tag myapp:1.0 myuser/myapp:1.0
docker push myuser/myapp:1.0

# Run a local private registry
docker run -d -p 5000:5000 --name registry registry:2
docker tag myapp:1.0 localhost:5000/myapp:1.0
docker push localhost:5000/myapp:1.0

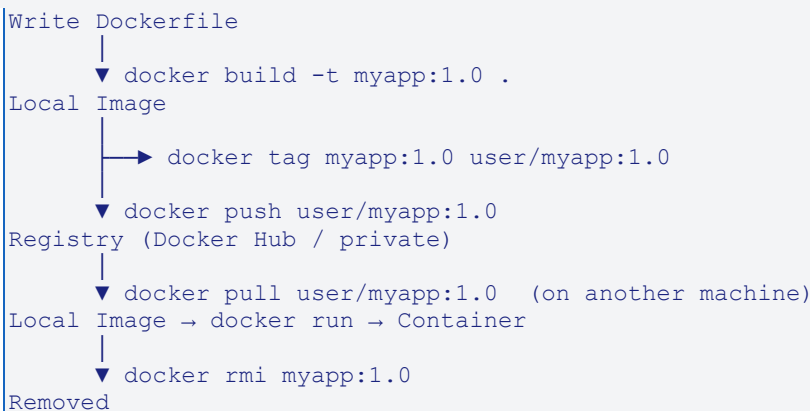
# Search Docker Hub
docker search nginx --limit 5

# Logout
docker logout
```

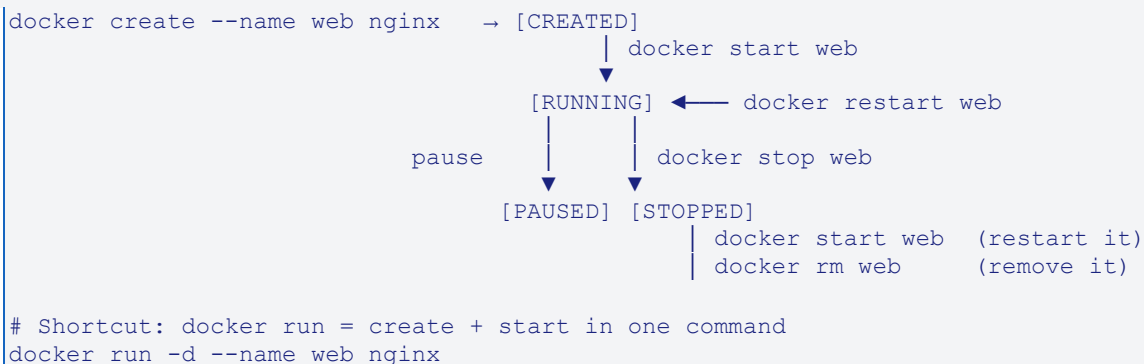
⚠ Never pull :latest in production — always pin a specific version tag. Never push images with secrets baked in. Always scan before pushing: `docker scout cves myimage`

3. WORKFLOW & COMMANDS

3.1 Image Lifecycle



3.2 Container Lifecycle



3.3 Essential CLI Quick Reference

System Commands

Command	Description
<code>docker version</code>	Client and Server version info
<code>docker info</code>	System-wide info (containers, images, storage)
<code>docker system df</code>	Disk usage by Docker (images, containers, volumes)
<code>docker system df -v</code>	Verbose disk usage breakdown
<code>docker system events</code>	Real-time stream of Docker events
<code>docker system prune</code>	Remove all unused resources
<code>docker system prune -a --volumes</code>	Remove EVERYTHING unused including volumes

docker run — Most Used Flags

Flag	Purpose	Example
-d	Detached mode (background)	docker run -d nginx
-it	Interactive + TTY (terminal)	docker run -it ubuntu bash
--name	Assign a name	docker run --name web nginx
-p	Publish ports	docker run -p 8080:80 nginx
-v	Mount volume or bind mount	docker run -v data:/app nginx
-e	Set environment variable	docker run -e PORT=3000 myapp
--env-file	Load env from file	docker run --env-file .env myapp
--network	Connect to network	docker run --network mynet nginx
--rm	Auto-remove on stop	docker run --rm alpine echo hi
--restart	Restart policy	docker run --restart=always nginx
--cpus	CPU limit	docker run --cpus="1.5" myapp
--memory	Memory limit	docker run --memory="512m" myapp

3.4 Docker Logs

```
# View all logs
docker logs web

# Follow logs in real time (like tail -f)
docker logs -f web

# Last N lines only
docker logs --tail 100 web

# Logs with timestamps
docker logs -t web

# Logs since a specific time / relative time
docker logs --since 2024-01-01T00:00:00 web
docker logs --since 30m web      # Last 30 minutes

# Combine: last 50 lines + follow + timestamps
docker logs -f --tail 50 -t web
```

3.5 Docker Inspect

docker inspect returns detailed JSON information about any Docker object (container, image, volume, network).

```
# Inspect a container
docker inspect web

# Extract specific field with Go template
docker inspect --format "{{.State.Status}}" web
docker inspect --format "{{.NetworkSettings.IPAddress}}" web
```

```
docker inspect --format "{{.Config.Env}}" web
docker inspect --format "{{.RestartCount}}" web

# IP across all networks
docker inspect -f "{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}" web

# Inspect an image
docker inspect nginx:1.25

# Inspect a volume / network
docker volume inspect mydata
docker network inspect bridge
```

3.6 Exec vs Attach

Feature	docker exec	docker attach
What it does	Runs a NEW process inside container	Connects to the MAIN process (PID 1)
Use case	Debugging, one-off commands	Viewing main process output
Exit behavior	Only exec process exits — container keeps running	Ctrl+C stops the main process = stops container!
Detach	Just exit normally	Press Ctrl+P, Ctrl+Q

```
# docker exec — ALWAYS prefer this for debugging
docker exec -it web bash      # Interactive shell
docker exec web cat /etc/hosts # Run single command
docker exec -u root web bash  # Run as specific user

# docker attach — only when you need main process I/O
docker attach web
# Detach without stopping: Ctrl+P then Ctrl+Q
```

⚠ Never use Ctrl+C with docker attach — it sends SIGINT to the main process and STOPS the container. Always use docker exec -it for interactive debugging.

3.7 Prune Commands — Cleaning Up

```
# Check disk usage first
docker system df

# Remove stopped containers
docker container prune

# Remove dangling (untagged) images only
docker image prune

# Remove ALL unused images
docker image prune -a

# Remove unused volumes
docker volume prune
```

```
# Remove unused networks
docker network prune

# Nuclear option: remove EVERYTHING unused
docker system prune
docker system prune -a --volumes # Including volumes (DANGER!)
```

⚠ docker volume prune and docker system prune --volumes delete volume data permanently. Always check docker system df first and know what you are deleting before confirming.

4. DOCKERFILE DEEP DIVE

 Source: docs.docker.com/reference/dockerfile/

4.1 All Major Instructions

FROM — Base Image

```
# Must be first instruction (except ARG)
FROM ubuntu:22.04
FROM python:3.11-slim
FROM node:20-alpine           # Minimal alpine variant
FROM scratch                  # Empty base (for static binaries)

# Pinned digest (immutable – most secure)
FROM python@sha256:abc123...

# Named build stage (for multi-stage builds)
FROM golang:1.21 AS builder
```

✓ Best Practice: Always use specific tags (python:3.11-slim). Never rely on :latest for reproducible production builds.

RUN — Execute Commands During Build

```
# Shell form
RUN apt-get update && apt-get install -y curl

# Exec form (no shell)
RUN ["apt-get", "install", "-y", "curl"]

# BEST PRACTICE: Chain with && and clean up in SAME layer
RUN apt-get update && \
    apt-get install -y --no-install-recommends \
        curl \
        wget \
    && rm -rf /var/lib/apt/lists/*
# The rm MUST be in the SAME RUN – else cache bloats the layer
```

⚠ Multiple separate RUN apt-get install commands create multiple layers and stale caches. Always chain with && in one RUN statement.

CMD — Default Runtime Command

```
# Sets the DEFAULT command – can be overridden at docker run

# Exec form (preferred)
CMD ["python", "app.py"]
CMD ["nginx", "-g", "daemon off;"]

# Override at runtime:
docker run myimage python other.py ← Overrides CMD

# Only the LAST CMD takes effect
```

ENTRYPOINT — Fixed Executable

```
# ENTRYPOINT cannot be easily overridden (need --entrypoint flag)
ENTRYPOINT ["python", "app.py"]

# BEST PATTERN: ENTRYPOINT + CMD for flexible defaults
ENTRYPOINT ["python"]
CMD ["app.py"]
# Default: python app.py
# docker run myimage other.py → python other.py      (CMD overridden)
# docker run --entrypoint bash myimage                (ENTRYPOINT overridden)
```

	CMD	ENTRYPOINT
Override	docker run myimg <cmd> overrides it	Requires --entrypoint flag to override
Use case	Default args or full default command	Fixed main executable
Combined	Provides default arguments to ENTRYPOINT	The main executable

COPY — Copy Files from Build Context

```
COPY requirements.txt /app/
COPY . /app/
COPY --chown=appuser:appgroup src/ /app/src/

# Copy from a previous build stage (multi-stage)
COPY --from=builder /app/binary /app/binary
```

ADD — Copy with Extras

```
# Like COPY but also:
ADD archive.tar.gz /app/      # Auto-extracts tar archives
ADD https://url/file /tmp/    # Fetch from URL (not recommended)

# RULE: Use COPY unless you specifically need these features
```

WORKDIR — Set Working Directory

```
WORKDIR /app
# All following RUN, CMD, ENTRYPOINT, COPY work from /app
# Creates directory if it does not exist

# Equivalent to: mkdir -p /app && cd /app
# ALWAYS use WORKDIR instead of RUN cd /app
```

ENV and ARG

```
# ENV — available at BUILD time AND at RUNTIME
ENV APP_ENV=production
ENV PORT=8080
```

```
# ARG — available at BUILD time ONLY (not in running container)
ARG PYTHON_VERSION=3.11
FROM python:${PYTHON_VERSION}-slim

# Pass ARG at build time:
docker build --build-arg PYTHON_VERSION=3.12 .

# Make ARG available at runtime by mapping to ENV:
ARG VERSION
ENV APP_VERSION=$VERSION

# NEVER put secrets in ENV or ARG — visible in docker inspect!
```

EXPOSE, VOLUME, HEALTHCHECK, USER

```
# EXPOSE — documentation only (does NOT publish the port)
EXPOSE 80
EXPOSE 443

# VOLUME — declare a mount point
VOLUME /data
VOLUME ["/data", "/logs"]

# USER — run as non-root (critical for security)
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser

# HEALTHCHECK — how Docker tests container health
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
  CMD curl -f http://localhost:8080/health || exit 1

# LABEL — metadata
LABEL maintainer="you@example.com" version="1.0"

# STOPSIGNAL — signal for graceful shutdown
STOPSIGNAL SIGTERM
```

4.2 Multi-Stage Builds

 Source: docs.docker.com/build/building/multi-stage/

Multi-stage builds use multiple FROM statements to create intermediate stages. Only final artifacts are copied to the production image — resulting in much smaller, more secure images.

```
# Without multi-stage: ~1.2 GB (node + npm + source + build tools)
# With multi-stage:    ~25 MB (nginx + built static files only)

# Stage 1: Build
FROM node:20 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build
```

```
# Stage 2: Production (only what is needed to run)
FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

```
# Go example - binary only in final image (~5MB)
FROM golang:1.21 AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -o myapp

FROM scratch # Empty - just the binary
COPY --from=builder /app/myapp /myapp
ENTRYPOINT ["/myapp"]
```

```
# Build only to a specific stage (useful for debugging)
docker build --target builder -t myapp:debug .
```

4.3 Build Cache Behaviour

```
# Cache rules:
# 1. If instruction unchanged → cache HIT
# 2. For COPY/ADD → file checksums compared
# 3. First cache MISS → ALL subsequent layers rebuild

# Optimal order: LEAST → MOST frequently changing
COPY package.json package-lock.json ./ # Rarely changes
RUN npm ci # Cached until package.json changes
COPY . . # Only this rebuilds on code change

# BuildKit cache mount - speed up package managers across builds
RUN --mount=type=cache,target=/root/.cache/pip \
    pip install -r requirements.txt

# Force rebuild of a specific layer
ARG CACHEBUST=1
RUN git clone https://github.com/user/repo.git
# docker build --build-arg CACHEBUST=$(date +%s) .
```

4.4 Best Practices & Security

Official Best Practices

1. Use `.dockerignore` to exclude `.git`, `node_modules`, `*.md`, `.env`, `__pycache__`
2. Minimise layers — combine related RUN commands with `&&`
3. Use multi-stage builds — keep production images lean
4. Use specific base image tags — never `:latest`
5. Order instructions for caching — least-changing first
6. Use `--no-install-recommends` with `apt-get`
7. Run as non-root user
8. Use `COPY` instead of `ADD` unless you need auto-extraction
9. One container = one process
10. Label images for organisation and automation

```
# .dockerignore file
.git
node_modules
*.md
.env
__pycache__
.vscode
dist
build
```

Security

```
# NEVER hardcode secrets
# BAD:
ENV API_KEY=sk-12345

# GOOD: pass at runtime or use Docker Secrets
docker run -e API_KEY=$API_KEY myapp

# Build-time secrets via BuildKit (never stored in layers)
# docker build --secret id=mysecret,src=./secret.txt .
RUN --mount=type=secret,id=mysecret cat /run/secrets/mysecret

# Scan images for vulnerabilities
docker scout cves myapp:1.0

# Use minimal base images
FROM python:3.11-slim # Not python:3.11
FROM node:20-alpine # Not node:20

# Pin package versions
RUN pip install flask==3.0.0
```

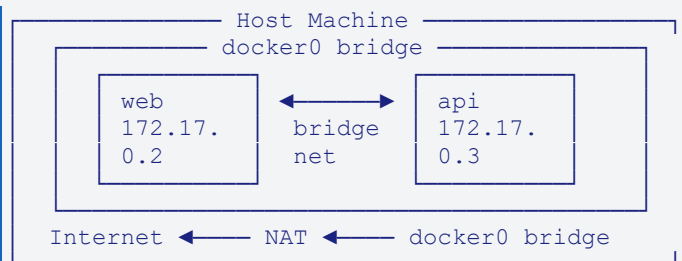
5. DOCKER NETWORKING

[Source: docs.docker.com/network/](https://docs.docker.com/network/)

5.1 Network Drivers Overview

Driver	Use Case
bridge	Default. Single-host container isolation + DNS
host	No network isolation. Container uses host network
overlay	Multi-host networking (Docker Swarm clusters)
macvlan	Container gets real MAC address on physical network
none	No networking. Maximum isolation.

5.2 Bridge Network



Feature	Default bridge	Custom bridge (recommended)
DNS by container name	No — IP only	Yes — automatic
Isolation	All containers share it	Only connected containers
Auto-connect	All containers by default	Explicit connection required
Connect/disconnect live	No	Yes

```

# Create custom bridge network (RECOMMENDED)
docker network create mynet

# Run containers on custom network
docker run -d --name db --network mynet postgres:16
docker run -d --name web --network mynet myapp

# Inside "web" container - DNS works by name:
# curl http://db:5432 ← resolves "db" via Docker DNS

# Inspect network
docker network inspect mynet
  
```

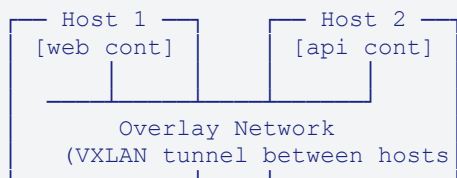
5.3 Host Network

```
# Container shares host network stack – no NAT
docker run -d --network host nginx
# nginx listens directly on host port 80 – no -p needed
```

Pros	Cons
Best performance (no NAT overhead)	No port isolation from host
Simplest for single-container setups	Port conflicts with host services
No port mapping needed	Linux only (not on Docker Desktop Mac/Windows)

5.4 Overlay Network (Swarm)

```
# Connects containers across multiple Docker hosts
```



```
# Initialize Swarm first
docker swarm init

# Create overlay network
docker network create --driver overlay my-overlay

# Deploy service on overlay
docker service create --name web --network my-overlay nginx
```

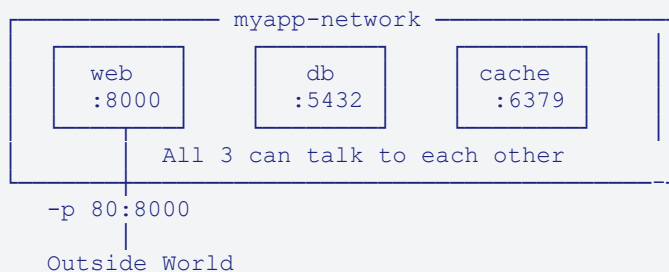
5.5 Macvlan Network

```
# Container gets real MAC address – appears as physical device
docker network create -d macvlan \
  --subnet=192.168.1.0/24 \
  --gateway=192.168.1.1 \
  -o parent=eth0 \
  my-macvlan

docker run -d --network my-macvlan --ip 192.168.1.100 nginx
# Use case: legacy apps that need to appear as physical hosts
```

5.6 Container-to-Container Communication

Pattern: Only "web" exposed externally, db and cache are private



```

docker network create myapp-network
docker run -d --name db --network myapp-network postgres:16
docker run -d --name cache --network myapp-network redis:7
docker run -d --name web --network myapp-network -p 80:8000 myapp:1.0
    
```

5.7 How DNS Works Inside Docker

Docker has a built-in DNS server on custom networks
Resolver address inside containers: 127.0.0.11

When "web" queries "db":
1. web sends DNS query to 127.0.0.11
2. Docker DNS resolves "db" → 172.18.0.3
3. Packet routed to db container

Test from inside a container:
docker exec -it web sh
nslookup db # Shows Docker DNS at 127.0.0.11
ping db # Works by name!

Network aliases (multiple names for same container)
docker run -d --name postgres --network mynet \
--network-alias db \
--network-alias database \
postgres:16
Reachable as "postgres", "db", OR "database"

⚠ DNS works ONLY on custom bridge networks. The default bridge network uses IP addresses only — this is why you should always create your own network.

6. DOCKER STORAGE

[Source: docs.docker.com/storage/](https://docs.docker.com/storage/)

6.1 Storage Types Overview

DOCKER STORAGE OPTIONS

Named Volumes Managed by Docker /var/lib/docker/ volumes/	Bind Mounts Host filesystem Any host path (you control it)	tmpfs In-memory only RAM only Not persisted
↓ Best for ↓ Production data Databases	↓ Best for ↓ Dev code sync Config files	↓ Best for ↓ Temp secrets Temp data

Feature	Named Volume	Bind Mount	tmpfs
Managed by	Docker	User / OS	Docker (RAM)
Persists after container rm	Yes	Yes (host file)	No
Location	/var/lib/docker/volumes/	Any host path	Memory only
Portability	High	Low (tied to path)	N/A
Performance	Good	OS-dependent	Fastest
Pre-populated from image	Yes	No	No
Best for	Databases, app data	Dev live reload	Sensitive temp data

6.2 Named Volumes in Depth

```
# Create
docker volume create pgdata

# Use (-v short form)
docker run -d --name postgres \
-v pgdata:/var/lib/postgresql/data \
-e POSTGRES_PASSWORD=secret \
postgres:16

# Use (--mount explicit form - recommended in scripts)
docker run -d --name postgres \
--mount type=volume,source=pgdata,target=/var/lib/postgresql/data \
-e POSTGRES_PASSWORD=secret \
postgres:16

# Data persists after container removal!
docker rm -f postgres
docker volume ls      # pgdata still exists

# Re-create container - all data intact
docker run -d --name postgres2 \
```

```
-v pgdata:/var/lib/postgresql/data \  
-e POSTGRES_PASSWORD=secret postgres:16  
  
# Inspect - see actual host path  
docker volume inspect pgdata  
# "Mountpoint": "/var/lib/docker/volumes/pgdata/_data"
```

Volume Backup & Restore

```
# Backup volume to tar.gz  
docker run --rm \  
-v pgdata:/data \  
-v $(pwd):/backup \  
alpine tar czf /backup/pgdata-backup.tar.gz -C /data .  
  
# Restore volume from backup  
docker run --rm \  
-v pgdata:/data \  
-v $(pwd):/backup \  
alpine tar xzf /backup/pgdata-backup.tar.gz -C /data
```

6.3 Bind Mounts in Practice

```
# Development: mount source code for live reload  
docker run -d --name dev-server \  
-v $(pwd)/src:/app/src \  
-p 3000:3000 \  
node-dev-server  
  
# Read-only config file mount  
docker run -d --name web \  
-v $(pwd)/nginx.conf:/etc/nginx/nginx.conf:ro \  
nginx  
  
# -v vs --mount:  
# -v /host:/container:ro - short form, creates dir if missing  
# --mount type=bind,source=..., - explicit, errors if host dir missing  
# target=...,readonly
```

6.4 Volume Drivers

```
# Default: local driver (host filesystem)  
docker volume create mydata  
  
# NFS volume driver  
docker volume create \  
--driver local \  
--opt type=nfs \  
--opt o=addr=192.168.1.100,rw \  
--opt device=/path/to/dir \  
nfs-volume  
  
# Use with container  
docker run -d -v nfs-volume:/data myapp
```

Driver	Storage Backend
local (default)	Local filesystem
nfs	Network File System
rexray / cloud plugins	AWS EBS, Azure Files, GCP Persistent Disk

6.5 Data Persistence Best Practices

11. Always use NAMED volumes for databases — never anonymous volumes
12. Backup volumes regularly before updates or migrations
13. Use read-only mounts for config files: `-v config:/etc/app:ro`
14. Never store important data only in the container writable layer — lost on `docker rm`
15. Use `.dockerignore` to avoid copying data directories into images
16. For development: bind mount source code + named volume for dependencies

⚠ Never run `docker volume prune` in production without verifying which volumes are "unused". An unused volume may still contain critical data from a stopped container.

7. DOCKER COMPOSE

[i Source: docs.docker.com/compose/](https://docs.docker.com/compose/)

7.1 What Problem Does Compose Solve?

```
# WITHOUT Compose - painful manual process
docker network create app-net
docker volume create pgdata
docker run -d --name db --network app-net \
  -v pgdata:/var/lib/postgresql/data \
  -e POSTGRES_PASSWORD=secret postgres:16
docker run -d --name redis --network app-net redis:7
docker run -d --name web --network app-net -p 8080:80 \
  -e DB_HOST=db -e REDIS_HOST=redis myapp

# WITH Compose - one command for entire stack
docker compose up -d
docker compose down
```

7.2 YAML Structure Breakdown

```
# compose.yaml

services:
  # Containers to run
  web:
    # Service name = DNS hostname within the stack
    image: nginx
    ports:
      - "8080:80" # host:container
    volumes:
      - ./html:/usr/share/nginx/html # Bind mount
    depends_on:
      - api
    networks:
      - frontend

  api:
    build: ./api # Build from Dockerfile in ./api/
    environment:
      - DB_HOST=db
    depends_on:
      db:
        condition: service_healthy # Wait for health check
    networks:
      - frontend
      - backend

  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: myapp
    volumes:
      - pgdata:/var/lib/postgresql/data # Named volume
    networks:
      - backend
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
```

```

    interval: 5s
    timeout: 5s
    retries: 5

volumes:      # Named volumes
  pgdata:

networks:     # Custom networks
  frontend:
  backend:
    internal: true  # No external internet access

```

7.3 Service Configuration Options

```

services:
  myservice:
    image: nginx:1.25          # Use existing image OR
    build:                     # Build from Dockerfile
      context: ./app
      dockerfile: Dockerfile.prod
    args:
      BUILD_ENV: production
    ports: ["8080:80"]
    expose: ["3000"]           # Internal only
    environment:
      DB_HOST: postgres
    env_file: [.env]
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./src:/app/src
      - ./config:/app/config:ro
    depends_on:
      db:
        condition: service_healthy
    deploy:
      resources:
        limits: {cpus: "1.0", memory: 512M}
        reservations: {cpus: "0.5", memory: 256M}
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost/"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 10s
    user: "1000:1000"
    working_dir: /app

```

7.4 Essential Compose Commands

Command	Description
<code>docker compose up -d</code>	Start all services in detached mode
<code>docker compose up -d --build</code>	Build images then start
<code>docker compose up --scale web=3</code>	Scale web service to 3 instances
<code>docker compose down</code>	Stop and remove containers + networks

Command	Description
<code>docker compose down -v</code>	Stop + remove containers + VOLUMES (DANGER)
<code>docker compose ps</code>	List running services and their status
<code>docker compose logs</code>	View logs from all services
<code>docker compose logs -f web</code>	Follow logs for specific service
<code>docker compose exec web bash</code>	Open shell in running service
<code>docker compose run --rm web python migrate.py</code>	Run one-off command in new container
<code>docker compose restart web</code>	Restart specific service
<code>docker compose pull</code>	Pull latest images for all services
<code>docker compose build</code>	Build or rebuild service images
<code>docker compose config</code>	Validate and view final resolved config
<code>docker compose stop</code>	Stop services without removing containers

7.5 Complete Example — Full-Stack Application

```
# compose.yaml - Blog Application (frontend + API + DB + cache)
services:
  frontend:
    build: ./frontend
    ports: ["3000:3000"]
    environment: {API_URL: "http://api:8000"}
    depends_on: [api]
    networks: [frontend-net]

  api:
    build: ./backend
    environment:
      DATABASE_URL: postgresql://user:pass@db:5432/blog
      REDIS_URL: redis://cache:6379
    depends_on:
      db: {condition: service_healthy}
      cache: {condition: service_started}
    volumes: [uploads:/app/uploads]
    networks: [frontend-net, backend-net]

  db:
    image: postgres:16-alpine
    environment: {POSTGRES_USER: user, POSTGRES_PASSWORD: pass, POSTGRES_DB: blog}
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U user -d blog"]
      interval: 5s
      retries: 5
    networks: [backend-net]

  cache:
    image: redis:7-alpine
    volumes: [redis-data:/data]
    networks: [backend-net]
```

```
volumes: {pgdata: {}, redis-data: {}, uploads: {}}
networks: {frontend-net: {}, backend-net: {}}
```

7.6 Compose V2 Features

Feature	Compose V1 (Legacy)	Compose V2 (Current)
Command	docker-compose (hyphen)	docker compose (no hyphen)
Language	Python	Go
Installation	Standalone binary	Docker CLI plugin
Preferred file	docker-compose.yml	compose.yaml
Performance	Slower	Faster

```
# Profiles - only start services when profile is active
services:
  web:
    image: nginx
  debug:
    image: busybox
    profiles: [debug]      # Only starts with --profile debug

docker compose up -d      # Starts web only
docker compose --profile debug up -d  # Starts web + debug
```

8. DOCKER SECURITY

[i Source: docs.docker.com/engine/security/](https://docs.docker.com/engine/security/)

8.1 Linux Namespaces

Docker uses Linux namespaces to provide isolation. Each container gets its own isolated view of the system.

Namespace	What It Isolates
PID	Process IDs — container sees only its own processes (PID 1 = main process)
NET	Network interfaces, IPs, routing tables, ports
MNT	Filesystem mount points (own root filesystem)
UTS	Hostname and domain name
IPC	Inter-process communication (shared memory, message queues)
USER	User and group IDs (UID/GID mapping)

8.2 Control Groups (cgroups)

cgroups limit and track the resources a container can use — CPU, memory, I/O, PIDs.

```
# CPU limits
docker run --cpus="1.5" myapp           # Max 1.5 CPU cores
docker run --cpu-shares=512 myapp       # Relative weight (default 1024)
docker run --cpuset-cpus="0,1" myapp    # Pin to specific cores

# Memory limits
docker run --memory="512m" myapp        # Hard limit
docker run --memory="512m" --memory-swap="1g" myapp

# PID limit
docker run --pids-limit=100 myapp       # Max 100 processes

# Monitor usage
docker stats
docker stats --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}"
```

8.3 Image Scanning

[i Source: docs.docker.com/scout/](https://docs.docker.com/scout/)

```
# Docker Scout — official built-in scanner
docker scout cves myapp:1.0
docker scout quickview myapp:1.0
docker scout recommendations myapp:1.0

# Fail CI/CD on vulnerabilities
docker scout cves --exit-code --only-severity critical,high myapp:1.0

# Trivy — popular open-source alternative
trivy image myapp:1.0
```

```
trivy image --severity HIGH,CRITICAL myapp:1.0
```

8.4 Least Privilege Principle

```
# In Dockerfile - create and switch to non-root user
FROM node:20-alpine
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
WORKDIR /app
COPY --chown=appuser:appgroup . .
USER appuser
CMD ["node", "server.js"]

# At runtime - additional hardening
docker run \
  --cap-drop ALL \           # Drop all Linux capabilities
  --cap-add NET_BIND_SERVICE \ # Add back only what is needed
  --read-only \             # Read-only filesystem
  --tmpfs /tmp \            # Writable tmpfs for /tmp
  --security-opt no-new-privileges \ # Prevent privilege escalation
  myapp

# NEVER use --privileged in production!
# It gives the container full access to the host!
```

⚠️ --privileged grants the container essentially root-level access to the host. Never use it in production unless you have an extremely specific need (like a container management tool).

8.5 Rootless Mode

i Source: docs.docker.com/engine/security/rootless/

```
# Run Docker daemon without root privileges
dockerd-rootless-setup.sh install

# Set environment
export PATH=$HOME/bin:$PATH
export DOCKER_HOST=unix://$XDG_RUNTIME_DIR/docker.sock

# Verify
docker info | grep "Root Dir"
# Should show a path under $HOME
```

- Benefit: Even if a container is compromised, attacker has no root on host
- Limitation: Cannot bind to ports < 1024 without extra configuration
- Limitation: Some storage drivers not supported

8.6 Docker Bench for Security

```
# Runs automated CIS Docker benchmark checks
docker run --rm --net host --pid host --usersns host \
  --cap-add audit_control \
  -v /etc:/etc:ro \
  -v /var/lib:/var/lib:ro \
  -v /var/run/docker.sock:/var/run/docker.sock:ro \
```

```
docker/docker-bench-security
```

```
# Reports: [PASS] [WARN] [INFO] for each check
# Checks: host config, daemon config, images, container runtime
```

8.7 Docker Content Trust (DCT)

i Source: docs.docker.com/engine/security/trust/

```
# Enable content trust - verify image signatures
export DOCKER_CONTENT_TRUST=1

# Pulls/pushes now require signed images
docker pull nginx          # Only succeeds if image is signed
docker push myapp:1.0      # Signs the image before pushing

# Disable for a specific command
DOCKER_CONTENT_TRUST=0 docker pull unsigned-image
```

8.8 Security Checklist

Practice	How to Implement
Run as non-root	USER appuser in Dockerfile
Drop capabilities	--cap-drop ALL --cap-add <needed>
Read-only filesystem	--read-only --tmpfs /tmp
No privileged mode	Never use --privileged
Scan images	docker scout cves <image>
Use minimal bases	alpine, slim, distroless variants
Pin versions	FROM python:3.11.7-slim (exact version)
Content trust	DOCKER_CONTENT_TRUST=1
Rootless mode	dockerd-rootless-setuptool.sh install
Audit	docker/docker-bench-security
Resource limits	--memory, --cpus flags
No secrets in images	Use --env-file or Docker Secrets

9. DOCKER IN PRODUCTION

[i Source: docs.docker.com/config/containers/](https://docs.docker.com/config/containers/)

9.1 Logging Drivers

[i Source: docs.docker.com/config/containers/logging/](https://docs.docker.com/config/containers/logging/)

Docker captures stdout and stderr from containers and routes them via pluggable logging drivers.

Driver	Description
json-file	Default. Writes JSON logs to disk. Set max-size/max-file!
syslog	Sends to syslog daemon
journald	Writes to systemd journal (Linux)
fluentd	Sends to Fluentd collector
awslogs	Amazon CloudWatch Logs
gcplogs	Google Cloud Logging
splunk	Splunk HTTP Event Collector
none	No logging — disables all log output

```
# Configure per-container logging
docker run -d \
  --log-driver=json-file \
  --log-opt max-size=10m \
  --log-opt max-file=3 \
  nginx

# Set globally in /etc/docker/daemon.json
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "5"
  }
}

# Check log driver of a container
docker inspect --format "{{.HostConfig.LogConfig.Type}}" web
```

⚠ Critical Production Tip: Always set max-size and max-file on json-file driver. Without them, logs grow unbounded and will eventually fill the host disk — causing container crashes.

9.2 Resource Limits

```
# Memory
docker run --memory="256m" myapp # Hard limit
docker run --memory="256m" --memory-swap="512m" myapp

# CPU
docker run --cpus="1.5" myapp # Max 1.5 cores
docker run --cpu-shares=512 myapp # Relative weight
```

```
docker run --cpuset-cpus="0,1" myapp    # Pin to cores 0 and 1

# PIDs (prevent fork bombs)
docker run --pids-limit=100 myapp

# In compose.yaml
deploy:
  resources:
    limits: {cpus: "1.0", memory: 512M}
    reservations: {cpus: "0.25", memory: 128M}
```

9.3 Restart Policies

 Source: docs.docker.com/config/containers/start-containers-automatically/

Policy	Behaviour
no (default)	Never restart
on-failure	Restart only if exit code is non-zero
on-failure:5	Restart on failure, max 5 attempts
always	Always restart (even after manual stop, if daemon restarts)
unless-stopped	Like always, but not after manual docker stop

```
docker run -d --restart=unless-stopped --name web nginx

# Update restart policy on existing container
docker update --restart=always web

# Check restart count
docker inspect -f "{{.RestartCount}}" web
```

✓ Production recommendation: Use unless-stopped for long-running services. Use on-failure for batch jobs that should retry but stop after a manual stop command.

9.4 Healthchecks

```
# In Dockerfile
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
  CMD curl -f http://localhost:8080/health || exit 1

# In compose.yaml
services:
  web:
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
      interval: 30s
      timeout: 5s
      retries: 3
      start_period: 10s

# Check health status
docker inspect --format "{{.State.Health.Status}}" web
# Values: starting | healthy | unhealthy
```

```
# View full health check log
docker inspect --format "{{json .State.Health}}" web | python -m json.tool
```

Health State Flow:

```

Container starts
  ↓
[starting] — start_period grace period →
  ↓
  ▼ (if checks pass)
[healthy]
  ↓
  ▼ (after "retries" consecutive failures)
[unhealthy] → restart policy kicks in (if configured)

```

9.5 Scaling Concepts

```
# Scale with Compose
docker compose up -d --scale web=3

# Scale with Swarm services
docker service scale myapp=5

# Rolling update with Swarm
docker service update \
  --image myapp:2.0 \
  --update-parallelism 2 \
  --update-delay 10s \
  myapp

# Rollback to previous version
docker service rollback myapp
```

9.6 Docker Swarm Basics

 Source: docs.docker.com/engine/swarm/

```
# Initialize Swarm (on manager node)
docker swarm init --advertise-addr <MANAGER-IP>

# Join as worker (on worker nodes)
docker swarm join --token <TOKEN> <MANAGER-IP>:2377

# View cluster nodes
docker node ls

# Create a replicated service
docker service create --name web --replicas 3 -p 80:80 nginx

# List services
docker service ls

# View service tasks (containers)
```

```
docker service ps web

# Scale service
docker service scale web=5

# Remove service
docker service rm web

# Deploy a whole stack (like Compose for Swarm)
docker stack deploy -c compose.yaml mystack
docker stack ls
docker stack services mystack
docker stack rm mystack
```

Concept	Description
Node	A Docker host in the Swarm cluster
Manager node	Orchestrates workload, maintains cluster state (Raft consensus)
Worker node	Runs containers assigned by manager
Service	Definition of tasks to run (replicated or global)
Task	A running container managed by Swarm
Ingress network	Built-in load balancing across nodes

10. ADVANCED TOPICS

10.1 BuildKit

i Source: docs.docker.com/build/buildkit/

BuildKit is Docker's next-generation build engine — default since Docker 23.0. It offers parallel builds, better caching, build secrets, SSH forwarding, and multi-platform support.

```
# Enable BuildKit (if not already default)
DOCKER_BUILDKIT=1 docker build -t myapp .

# Or in /etc/docker/daemon.json:
{ "features": { "buildkit": true } }

# Build secret — never stored in image layers
# Dockerfile:
RUN --mount=type=secret,id=npmmrc,target=/root/.npmmrc npm install
# Build command:
docker build --secret id=npmmrc,src=$HOME/.npmmrc -t myapp .

# SSH forwarding — clone private repos during build
# Dockerfile:
RUN --mount=type=ssh git clone git@github.com:user/private-repo.git
# Build command:
docker build --ssh default -t myapp .

# Cache mount — speed up package installs across builds
RUN --mount=type=cache,target=/root/.cache/pip \
    pip install -r requirements.txt

# Multi-platform builds
docker buildx create --name mybuilder --use
docker buildx build --platform linux/amd64,linux/arm64 \
    -t myapp:1.0 --push .
```

10.2 Docker Swarm — Extended

Service Types

Type	Description	Example
Replicated	Run N copies distributed across nodes	Web servers, APIs
Global	Run exactly one copy per node	Monitoring agents, log collectors

```
# Replicated service
docker service create --name web --replicas 3 nginx

# Global service (one per node)
docker service create --name monitor --mode global datadog/agent

# Swarm Secrets
echo "my-password" | docker secret create db_password -
docker service create --name web --secret db_password nginx
```

```
# Secret available at /run/secrets/db_password inside container

# Swarm Configs
docker config create nginx_conf ./nginx.conf
docker service create --name web --config nginx_conf nginx

# Promote worker to manager
docker node promote <node-id>

# Drain node before maintenance
docker node update --availability drain <node-id>
```

10.3 Private Docker Registry

 Source: docs.docker.com/registry/

```
# Run a local private registry
docker run -d -p 5000:5000 --name registry \
  -v registry-data:/var/lib/registry \
  registry:2

# Push and pull from it
docker tag myapp:1.0 localhost:5000/myapp:1.0
docker push localhost:5000/myapp:1.0
docker pull localhost:5000/myapp:1.0

# List images in registry (API)
curl http://localhost:5000/v2/_catalog

# Secure with TLS (production)
docker run -d -p 5000:5000 \
  -v /certs:/certs \
  -e REGISTRY_HTTP_TLS_CERTIFICATE=/certs/domain.crt \
  -e REGISTRY_HTTP_TLS_KEY=/certs/domain.key \
  registry:2
```

10.4 Docker Context

 Source: docs.docker.com/engine/context/working-with-contexts/

Docker contexts let you switch between multiple Docker endpoints — local, remote servers, or cloud.

```
# List contexts
docker context ls

# Create context for remote server (via SSH)
docker context create remote-server \
  --docker "host=ssh://user@remote-server"

# Switch to context — ALL commands now target remote
docker context use remote-server
docker ps    # Shows remote containers!

# Switch back to local
docker context use default

# One-time remote command without switching
docker --context remote-server ps
```

```
# Remove context
docker context rm remote-server
```

10.5 Docker Extensions

[Source: docs.docker.com/desktop/extensions/](https://docs.docker.com/desktop/extensions/)

Docker Extensions add functionality to Docker Desktop through a marketplace. Popular extensions include:

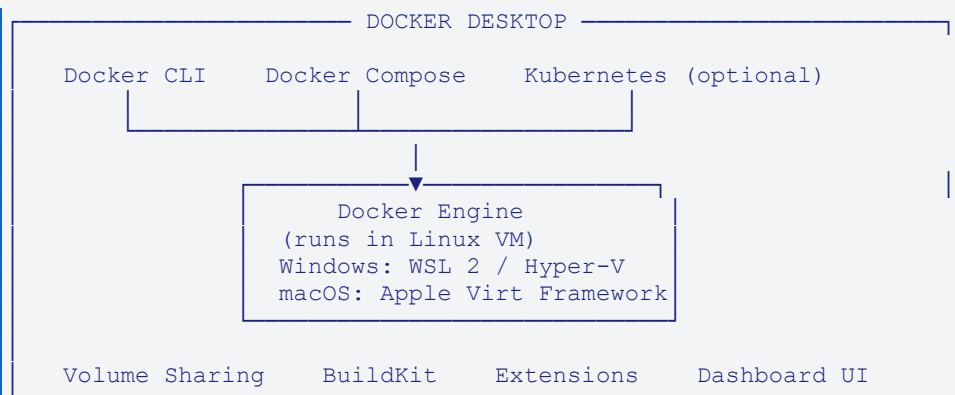
- Disk Usage — visualise what is consuming Docker disk space
- Logs Explorer — advanced log viewer with filtering
- Snyk / Scout — vulnerability scanning in the UI
- Portainer — full container management web UI
- Tailscale — private networking across machines

```
# Install an extension via CLI
docker extension install portainer/portainer-docker-extension

# List installed extensions
docker extension ls

# Remove an extension
docker extension rm portainer/portainer-docker-extension
```

10.6 Docker Desktop Architecture



✓ On Windows and macOS, Docker containers run inside a lightweight Linux VM. This is why host networking behaves differently — the "host" is the VM, not your Mac/Windows machine.

11. TROUBLESHOOTING

11.1 Common Errors & Fixes

Error: "port is already allocated"

```
Error: driver failed programming external connectivity:
Bind for 0.0.0.0:8080 failed: port is already allocated
```

```
# Find what is using the port
lsof -i :8080           # Linux/Mac
netstat -ano | findstr :8080 # Windows
```

```
# Use a different host port
docker run -p 8081:80 nginx
```

Error: "no space left on device"

```
# Check Docker disk usage
docker system df
```

```
# Clean up step by step
docker container prune
docker image prune -a
docker builder prune
docker volume prune      # CAREFUL - deletes volume data
```

```
# Nuclear (careful!)
docker system prune -a --volumes
```

```
# Check largest images
docker images --format "{{.Size}}\t{{.Repository}}:{{.Tag}}" | sort -rh | head -20
```

Error: "image not found" / "manifest unknown"

```
# Check image name and tag spelling
docker search <image-name>
```

```
# Log in if private registry
docker login
```

```
# Verify tag exists on Docker Hub: hub.docker.com/r/<user>/<image>/tags
```

Error: Container exits immediately

```
# Check exit code and logs
docker ps -a      # See STATUS column - exit code shown
docker logs <container> # What did the process output?
```

```
# Exit code meanings:
# 0   → process completed normally (fine for one-shot commands)
# 1   → application error (check logs)
```

```
# 137 → OOM killed (increase memory limit)
# 139 → segfault
# 143 → SIGTERM (normal graceful shutdown)

# Debug by overriding entrypoint
docker run -it --entrypoint sh myimage
# Now manually run the CMD and observe the error
```

Error: "Cannot connect to the Docker daemon"

```
# Linux: Start Docker daemon
sudo systemctl start docker
sudo systemctl enable docker    # Start on boot

# Windows/Mac: Start Docker Desktop application

# Check status
sudo systemctl status docker

# Permission issue? Add user to docker group
sudo usermod -aG docker $USER
# Log out and back in
```

Error: Build context too large

```
# Seen as: Sending build context to Docker daemon 2.5GB

# Create / update .dockerignore
.git
node_modules
*.md
.env
__pycache__
dist
build
*.tar.gz
```

Error: Containers cannot communicate

```
# Check if on same network
docker inspect web --format "{{.NetworkSettings.Networks}}"
docker inspect db  --format "{{.NetworkSettings.Networks}}"

# Test DNS from inside container
docker exec -it web sh
ping db
nslookup db

# Fix: put both on same CUSTOM network
docker network create app-net
docker network connect app-net web
docker network connect app-net db
```

Error: exec format error

```
# exec /usr/bin/myapp: exec format error
# Cause: image built for different CPU architecture

# Check image architecture
docker image inspect myapp | grep Architecture

# Build for correct platform
docker buildx build --platform linux/amd64 -t myapp .
```

11.2 Debugging Workflow

```
Container not working?
  ↓
  ▼ docker ps -a
1. Check status & exit code
  ↓
  ▼ docker logs <container>
2. Check application logs
  ↓
  ▼ docker inspect <container>
3. Inspect config (env, mounts, network, restart count)
  ↓
  ▼ docker exec -it <container> sh
4. Get interactive shell inside
  ↓
  ▼ docker network inspect <net>
5. Check network / connectivity
  ↓
  ▼ docker stats
6. Check resource usage (OOM?)
  ↓
  ▼ docker system events
7. Check Docker-level events (real time)
```

Command	Description
<code>docker top <container></code>	See all processes running inside container
<code>docker stats</code>	Real-time CPU / memory usage
<code>docker diff <container></code>	See filesystem changes (A=added, C=changed, D=deleted)
<code>docker export -o fs.tar <c></code>	Export container filesystem for external inspection
<code>docker system events</code>	Real-time stream of Docker daemon events

11.3 Performance Optimisation Tips

Image Size

17. Use alpine or slim base images (node:20-alpine = ~50MB vs ~350MB)
18. Use multi-stage builds — exclude build tools from production image
19. Minimize layers — combine RUN commands with && and clean up in same layer
20. Always use .dockerignore to reduce build context sent to daemon

Build Speed

21. Order Dockerfile correctly — dependencies before code for cache hits
22. Enable BuildKit: `DOCKER_BUILDKIT=1 docker build .`
23. Use cache mounts: `RUN --mount=type=cache,target=/root/.cache/pip pip install`
24. Parallelize stages in multi-stage builds with BuildKit

Runtime Performance

25. Set appropriate resource limits — `--cpus`, `--memory`
26. Use `--network host` when latency matters and security allows
27. Use `--tmpfs` for temporary data (in-memory, much faster than disk)
28. Monitor continuously: `docker stats --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}"`

12. INTERVIEW PREPARATION

12.1 Beginner-Level Questions

Q1: What is Docker and why do we use it?

i Answer: Docker is an open platform for developing, shipping, and running applications in containers. Containers package an app with all its dependencies so it runs consistently across all environments — solving the "works on my machine" problem. Benefits: fast startup, lightweight, portable, consistent environments.

Q2: Containers vs Virtual Machines?

Aspect	Virtual Machine	Container
OS	Full guest OS per VM (GBs)	Shares host OS kernel (MBs)
Startup	Minutes	Seconds
Isolation	Hardware-level (hypervisor)	Process-level (namespaces)
Use case	Full OS isolation needed	Microservices, CI/CD, scaling

Q3: What is a Dockerfile?

A text file with step-by-step instructions to build a Docker image. Each instruction creates a layer. Docker reads it top to bottom during docker build.

Q4: Difference between CMD and ENTRYPOINT?

CMD provides the default command/arguments that can be overridden when running `docker run myimage <override>`. ENTRYPOINT sets the fixed main executable and requires `--entrypoint` to override. When combined: ENTRYPOINT is the executable, CMD provides default arguments to it.

Q5: COPY vs ADD?

Both copy files from host to image. ADD can also auto-extract tar archives and download from URLs. Best practice: use COPY unless you specifically need those ADD features — COPY is more explicit and predictable.

Q6: What are Docker volumes?

Docker-managed persistent storage that survives container deletion. Stored at `/var/lib/docker/volumes/` on the host. Preferred for production data (especially databases) because they are portable, backed by Docker, and can be pre-populated from the container image.

12.2 Intermediate-Level Questions

Q7: Explain Docker networking — what are the drivers?

- Bridge (default): Isolated network on single host. Custom bridges enable DNS by container name.
- Host: Container shares host network stack. Best performance, no isolation.
- Overlay: Multi-host networking for Swarm clusters (VXLAN tunnel).
- Macvlan: Container gets real MAC address — appears as physical device on network.

Q8: What are layers and how does caching work?

Each Dockerfile instruction creates a read-only layer. Docker caches layers and reuses them if the instruction and its inputs are unchanged. Once a cache MISS occurs, all subsequent layers rebuild. Key optimisation: order instructions from least-changing to most-changing.

Q9: docker stop vs docker kill?

docker stop sends SIGTERM, waits 10 seconds for graceful shutdown, then sends SIGKILL. docker kill sends SIGKILL immediately with no grace period. Always prefer docker stop — databases need the grace period to flush writes.

Q10: What is a multi-stage build?

Using multiple FROM statements in a Dockerfile. Build tools and dependencies stay in early stages; only final artifacts are copied to the lean production stage via COPY --from=stagenam. A Go binary image shrinks from ~1GB to ~5MB.

Q11: How do containers communicate?

Containers on the same custom bridge network can communicate by container name via Docker's embedded DNS server at 127.0.0.11. The default bridge has NO DNS — IP addresses only. For multi-host communication, use overlay networks with Swarm.

Q12: What is Docker Compose?

A tool for defining and running multi-container applications using a YAML file (compose.yaml). One docker compose up command creates all networks, volumes, and starts all services in the correct dependency order.

12.3 Advanced-Level Questions

Q13: How does Docker achieve isolation?

- Namespaces: Isolate PID (processes), NET (network), MNT (filesystem), UTS (hostname), IPC, USER (UIDs)
- cgroups: Limit CPU, memory, I/O, PIDs per container
- Union filesystem (overlay2): Layered filesystem with copy-on-write — each container gets a thin writable layer on top of read-only image layers

Q14: How would you troubleshoot a container that keeps restarting?

29. docker ps -a — check exit code
30. docker logs — check application errors
31. docker inspect — check config, env vars, restart count
32. docker stats — check if OOM killed (exit code 137)
33. docker events — check Docker-level events
34. docker run -it --entrypoint sh myimage — debug interactively

Q15: How do you secure Docker in production?

- Run as non-root user (USER instruction in Dockerfile)
- Drop capabilities (--cap-drop ALL)
- Read-only filesystem (--read-only)
- Scan images (docker scout cves)

- Use minimal base images (alpine/distroless)
- Never use `--privileged`
- Enable content trust (`DOCKER_CONTENT_TRUST=1`)
- Set resource limits (`--memory`, `--cpus`)
- Use rootless mode

Q16: What is BuildKit?

Docker's next-generation build engine (default since Docker 23.0). Key benefits: parallel stage building, better caching, build-time secrets that are never stored in image layers (`--mount=type=secret`), SSH forwarding during builds, cache mounts for package managers, and multi-platform builds via `docker buildx`.

12.4 Scenario-Based Problems

Scenario 1: App works locally, crashes in Docker

```
# Diagnose
docker logs myapp           # Check error message
docker exec -it myapp sh    # Shell in and investigate
docker exec -it myapp env    # Check env vars are correct

# Common causes:
# 1. node_modules compiled for wrong architecture (native modules)
# 2. Missing env var (works locally because of .env file, not in container)
# 3. File path differences (case sensitivity on Linux vs Mac)
# Fix: Ensure all env vars are passed via -e or --env-file
```

Scenario 2: Container killed with exit code 137

```
# 137 = OOM killed (out of memory)

# Confirm
docker inspect <container> | grep -i OOMKilled
docker stats --no-stream

# Fix: Increase memory limit
docker run --memory="1g" --memory-swap="2g" myapp

# Or fix the memory leak in your application
```

Scenario 3: Docker build is very slow

```
# Check build context size (shown at start of docker build output)
# "Sending build context to Docker daemon 2.5GB" ← too big!

# Fix 1: Add .dockerignore
# Fix 2: Reorder Dockerfile - COPY package.json before COPY . .
# Fix 3: Enable BuildKit
DOCKER_BUILDKIT=1 docker build .
# Fix 4: Use cache mounts
RUN --mount=type=cache,target=/root/.cache/pip pip install -r req.txt
```

Scenario 4: Data lost after container restart

```
# Problem: data was in container writable layer – deleted with container

# Fix: use named volume
docker volume create myapp-data
docker run -v myapp-data:/app/data myapp

# Verify volume persists after container removal
docker rm -f myapp
docker volume ls      # myapp-data still there!
docker run -v myapp-data:/app/data myapp    # data is intact
```

Scenario 5: High CPU usage — diagnosis

```
# Step 1: Identify culprit container
docker stats --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}"

# Step 2: See processes inside
docker top <container>

# Step 3: Shell in and use top
docker exec -it <container> sh
top

# Step 4: Limit CPU while fixing root cause
docker update --cpus="1.0" <container>
```

13. 30-DAY MASTERY ROADMAP & REFERENCES

30-Day Docker Mastery Roadmap

Week 1 — Foundation (Days 1–7)

Day	Topic	Hands-On Practice
1	Install Docker, what is Docker, architecture	Run <code>docker run hello-world</code> , explore Docker Desktop UI
2	Images & Containers basics	Pull 5 images, run/stop/remove containers
3	Docker CLI mastery	Practice <code>ps</code> , <code>logs</code> , <code>exec</code> , <code>inspect</code> , <code>cp</code> , <code>stats</code>
4	Write your first Dockerfile	Dockerise a simple app in any language
5	Dockerfile instructions deep dive	Practice <code>FROM</code> , <code>RUN</code> , <code>CMD</code> , <code>COPY</code> , <code>ENTRYPOINT</code> , <code>WORKDIR</code>
6	Volumes & Bind Mounts	Run Postgres with named volume, bind mount for live reload
7	Review & consolidate	Redo Days 1–6 from memory without notes

Week 2 — Networking & Compose (Days 8–14)

Day	Topic	Hands-On Practice
8	Docker Networking (bridge, host)	Create custom network, run 2 containers that talk by name
9	Ports & Environment Variables	Map ports, pass env vars, use <code>--env-file</code>
10	Docker Compose intro	Write <code>compose.yaml</code> for a 2-service app (web + db)
11	Compose deep dive	Add healthchecks, <code>depends_on</code> , volumes, networks
12	Full-stack Compose project	Build 3-service app: frontend + API + database
13	Layers, cache & <code>.dockerignore</code>	Optimise Dockerfile for cache. Compare image sizes.
14	Review & Practice	Build a complete project from scratch with Compose

Week 3 — Production & Security (Days 15–21)

Day	Topic	Hands-On Practice
15	Multi-stage builds	Refactor project to multi-stage. Compare image sizes.
16	Security: non-root, capabilities	Add <code>USER</code> , <code>--cap-drop</code> , <code>--read-only</code> to your project
17	Image scanning	Scan with <code>docker scout cves</code> , fix vulnerabilities
18	Logging & Resource limits	Configure log rotation, set memory/CPU limits
19	Restart policies & Healthchecks	Add <code>restart: unless-stopped</code> and healthchecks to Compose
20	Private Registry	Run local registry, push/pull images to it

Day	Topic	Hands-On Practice
21	Review & Practice	Secure and optimise your full project

Week 4 — Advanced & Interview (Days 22–30)

Day	Topic	Hands-On Practice
22	BuildKit features	Cache mounts, build secrets, multi-platform builds
23	Docker Swarm basics	Init Swarm, create services, scale, rolling update
24	Overlay networks & Stacks	Deploy a stack across Swarm nodes
25	Docker Context	Set up contexts for local and remote endpoints
26	Troubleshooting practice	Debug broken containers using the workflow from Section 11
27	Performance optimisation	Reduce image sizes, speed up builds, tune runtime
28	Interview prep: Beginner/Intermediate	Answer all questions from Section 12 from memory
29	Interview prep: Advanced scenarios	Work through all scenario-based problems
30	Final Project	Build, secure, optimise, and deploy production-ready multi-container app

Practical Project Ideas

35. Personal Blog: WordPress + MySQL using Compose with persistent volumes
36. REST API: FastAPI or Express + PostgreSQL + Redis cache layer
37. CI Runner: Jenkins in Docker with Docker-in-Docker for builds
38. Monitoring Stack: Prometheus + Grafana + cAdvisor all in Compose
39. Microservices: 3 services behind Nginx reverse proxy deployed to Swarm

Official Documentation Links

Getting Started

Topic	URL
Docker Overview	docs.docker.com/get-started/overview/
Get Docker	docs.docker.com/get-docker/
Getting Started Guide	docs.docker.com/get-started/

Core References

Topic	URL
Dockerfile Reference	docs.docker.com/reference/dockerfile/
Docker CLI Reference	docs.docker.com/reference/cli/docker/
Docker Compose Reference	docs.docker.com/compose/compose-file/
Docker Hub	hub.docker.com

Building

Topic	URL
Build Best Practices	docs.docker.com/build/building/best-practices/
Multi-Stage Builds	docs.docker.com/build/building/multi-stage/
BuildKit	docs.docker.com/build/buildkit/
Build Cache	docs.docker.com/build/cache/

Networking, Storage & Security

Topic	URL
Networking Overview	docs.docker.com/network/
Volumes	docs.docker.com/storage/volumes/
Bind Mounts	docs.docker.com/storage/bind-mounts/
Security Overview	docs.docker.com/engine/security/
Rootless Mode	docs.docker.com/engine/security/rootless/
Content Trust	docs.docker.com/engine/security/trust/
Docker Scout	docs.docker.com/scout/

Production & Orchestration

Topic	URL
Logging Drivers	docs.docker.com/config/containers/logging/
Restart Policies	docs.docker.com/config/containers/start-containers-automatically/
Resource Constraints	docs.docker.com/config/containers/resource_constraints/
Docker Swarm Overview	docs.docker.com/engine/swarm/
Stack Deploy	docs.docker.com/engine/swarm/stack-deploy/
Docker Context	docs.docker.com/engine/context/working-with-contexts/
Docker Extensions	docs.docker.com/desktop/extensions/

Quick Reference Card

Command	Description
<code>docker build -t name:tag .</code>	Build image from Dockerfile
<code>docker run -d -p 8080:80 --name web nginx</code>	Run container in background with port mapping
<code>docker ps -a</code>	List ALL containers (running + stopped)
<code>docker logs -f web</code>	Follow container logs in real time
<code>docker exec -it web bash</code>	Open interactive shell inside container
<code>docker stop web</code>	Graceful stop (SIGTERM then SIGKILL)
<code>docker rm web</code>	Remove stopped container
<code>docker images</code>	List all local images
<code>docker pull nginx:1.25</code>	Download specific image version
<code>docker push user/image:tag</code>	Push image to registry
<code>docker network create mynet</code>	Create custom bridge network
<code>docker volume create mydata</code>	Create named volume
<code>docker compose up -d</code>	Start entire stack from compose.yaml
<code>docker compose down -v</code>	Stop stack and remove volumes
<code>docker system prune -a</code>	Remove all unused resources (careful!)
<code>docker inspect web</code>	Full JSON details of container
<code>docker stats</code>	Live CPU and memory usage
<code>docker scout cves image:tag</code>	Scan image for vulnerabilities
<code>docker system df</code>	Show Docker disk usage summary
<code>docker builder prune</code>	Clean build cache



Happy Dockering! — All content sourced from docs.docker.com